288 Assignment 3 Report: Nils Lund, Chris Jin, Aidan Wen, Hanjun Yoo

Q1: QA Data Creation

We built a 200 question dataset based on crawling data from the EECS website.We intentionally limited the QA to one question per webpage to maximize breadth. We filtered based on the prefix of urls, fetching more urls based on more important topics. We creatively used 4 different methods to generate QA as listed below. Note that we have a combined 5% non-extractive questions to challenge the RAG.

57%: *Specific factoids*. Testing precise entity extraction.
38%: *Dense retrieval.* Requires locating answers buried within long webpages or paragraphs.
3%: *Abstractive counting.* These require tallying unnumbered lists or doing simple arithmetic.
2%: *Logical Boolean*. The model must reason over retrieved information in order to answer Y/N.

In order to confirm dataset quality, we annotated 30% of the dataset (60 questions), achieving 85% exact-match IAA (excluding punctuation and capitalization). Most disagreements in the annotations were due to minor semantic differences, resolved by allowing both answers.

Type	Questions (Abbreviated)	Answer & Notes
Specific Factoid	What system is listed for internships, job postings, competitions, and recruiting events for current EECS and CS students?	**A:** "Handshake" ***Note:*** Example of easy QA. Answer appears on multiple pages of the corpus.
Specific Factoid	Which award did Richard Karp and coauthors receive in 2017?	**A:** "RECOMB Test of Time Award" ***Note****:* Corpus lists full conference name. RAG must abbreviate to hit 10-word limit.
Counting	How many application materials are listed for the A.C. to review when applying to the EECS/CS M.S. from another department?	**A:** "4" ***Note****:* "4" is not on the page. RAG must count the 4 requirements (transcripts, letters, statement).

Q2. Retrieval Corpus

We built our retrieval corpus using urllib.request and BeautifulSoup via BFS, filtering out tags commonly associated with repetitive content and converting HTML tables to Markdown. Key features include: excluding sparse pages, exponential backoff, retry mechanisms for timeouts, and HTTPS conversion. We evaluated by manually comparing random pages to the original website across multiple dev rounds.

After receiving the staff solution corpus, we ran ablations comparing the two datasets. During the test all hyperparameters remained the same, only the corpus source differed. We measured F1 score using the Gradescope portal to test these two RAGs on the official Dev and Test QA sets.

Our solution performed marginally better on both sets. Using our corpus, the RAG achieved 61.7% F1 on Dev and 65.7% F1 on test. The staff corpus achieved 61.4% F1 on Dev and 65.5% F1 on test. This is notable given that the staff QA set was likely generated from the staff corpus, giving it a natural advantage, yet our corpus performs better.

Q3. RAG System Architecture

Corpus ingestion: Markdown of dev corpus converted to plain text with section headings preserved as inline labels. Then chunked at 160 words with 20-word overlap.
Indexing: Chunks indexed with BM25 and FAISS (Full dense, bge-small-en-v1.5). Each chunk's indexed text is prefixed with page title and section heading to provide topic context. We also tested bge-base-en-v1.5 (110M parameters) but saw no meaningful improvement.
Retrieval: Hybrid BM25 + dense retrieval fused via RRF (Using rank of two metrics, instead of raw value). Weights for BM25 and dense equal. Query expansion (2 LLM-generated rewrites of query) and HyDE (Rewritten into answer passage-like) run in parallel at query time. Top 30 chunks selected.
Reranking: Cross-encoder (bge-reranker-base) rescores all 30 (chunk, query) pairs; top 10 returned.
Generation: Passed to Llama-3.1-8B-Instruct. Runs multiple times for Self-consistency to reduce generation variance. Reorder ranks to place highest-scored chunks at the start and end of the context window instead of in the middle, for better LLM context preservation. We also tested alternative LLMs but found Llama-3.1-8B-Instruct performed best.

Post-processing: Rule-based answer cleanup strips “Answer:”-like prefixes, punctuation. Uses dev set answers as reference to cut answer down to answer format that gradescope expects, for better F1.
Parent-child chunking: 160 word chunks for precise retrieval, expands to 500-word parent window for generation, provide more context in LLM.
URL Deduplication: Caps the number of chunks in single page in the reranked set, preventing a single page from dominating the context window.

Q4. Ablation Studies

Ablation 1: Moving from no reranker to a cross-encoder reranker produced the largest single jump in our experiments. 49.3 / 59.1 → 62.7 / 66.7 (+13.4 dev, +7.6 test). Note that chunk size also decreased (150→100) and retrieval width increased (20→40) alongside the reranker, so this ablation captures a combined effect. Nevertheless, retrieving a large candidate pool and then using a cross-encoder to precisely rerank a smaller final set seems to perform well, catching relevance signals that the bi-encoder's independent embeddings miss.
Ablation 2: Increasing the dense retrieval weight improved test-set performance. Additionally, comparing Top-K 80 retrieve / 15 rerank (68.1 / 68.9) against Top-K 40 retrieve / 10 rerank (61.7 / 65.7) shows that a wider retrieval pool with a tighter reranked set outperforms narrower retrieval. This suggests the test set contains more questions requiring semantic matching across paragraph boundaries. Dense retrieval handles paraphrase and conceptual similarity better than BM25's exact term matching too. This also explains why URL deduplication helps on test more than dev: it prevents large pages from crowding out the semantically relevant passages that dense retrieval gets.

Q5. Error Analysis

Most errors are answer-mapping failures. Many “incorrect” answers are false negatives caused by exact match metric limitations rather than genuine answer failures. A large share comes from normalization or granularity mismatches, where the model retrieves correct information but returns it in the wrong form (ex. “Three.” vs. “3”). We propose a better evaluation with LLM-as-judge to catch these false negatives.

Another category is counting/aggregation error, when the correct page is found but the model copies local details rather than computing the requested total. Examples include answering “M.A. ’05, Ph.D. ’07” instead of “2,” or giving “three items” when the correct number of application materials is “4”. This may partly reflect the limited reasoning capacity of a relatively small model, especially for questions that require lightweight multi-step aggregation such as calculation rather than direct span extraction.

We also see field extraction and entity confusion errors, where the model selects a nearby but incorrect entity from a closely related context, such as predicting “Scott Shenker” instead of “Ion Stoica” for the Fiat Lux Faculty Award or “Jiantao Jiao” instead of “Rishabh Iyer” for CS 168 Spring 2026.

Finally, a smaller set reflects outright retrieval failure, when the RAG outputs “UNKNOWN” to questions whose correct answers exist within the data (ex, “John F. Canny,” “2018,” and “1983.”)

Q6. Takeaways & Future Ideas

The assignment reinforced that retrieval quality dominates end-to-end RAG performance: our largest gains came from implementing a cross-encoding reranker rather than generation improvements. In ablation, our corpus performed marginally better than the reference on the staff QA. Notably, the staff file name (“rewritten”) suggests they used LLM document rewriting, which we did not – yet both solutions reached a similar performance plateau, suggesting diminishing returns from corpus preprocessing alone. Components like URL deduplication and parent-child chunking showed mixed results across dev and test sets, highlighting how individual design choices can trade off differently across evaluation splits. Throughout development, we were constrained by a 2400-second runtime window, 4GB RAM, and no GPU access. This made LLM-heavy components like reranking, HyDE, and query expansion the primary bottlenecks, limiting how aggressively we could scale these. With more time and resources, we would explore: (1) dense-only retrieval with a larger bi-encoder; (2) iterative retrieval, where the model retrieves, reads, then issues a refined query; (3) answer-grounded reranking, where the reranker scores chunks against a hypothetical answer rather than the question; and (4) training a lightweight span-extraction head on top of the retrieved context instead of relying solely on an instruction-tuned LLM.

Extra Pages

Contribution Statement

Hanjun contributed to the crawling starter code and debugging crawling, Aidan contributed to dataset creation (crawling) via running, debugging, and iterative improvements. Aidan and Hanjun contributed to dataset annotation. Chris and Aidan contributed to test set creation, all four members wrote the report (Nils: Q6, Extra Pages, Chris: Q3, Q5, Aidan: Q1, Q2, Hanjun: Q4). Aidan served as primary report editor. Nils, Chris and Hanjun contributed to the base of the rag pipeline code, with Nils contributing the most. All four members contributed to iterative improvements to the rag pipeline in attempts to achieve higher gradescope scores, with Nils, Hanjun, Chris and Aidan in decreasing order of contribution. Ablation and Error analysis were done by Hanjun, Chris, and Aidan, with Chris contributing the most due to his work on ablation scripting. Aidan put in the most effort into writing the report and bringing everything together into a coherent final version.

GenAI Acknowledgement

I acknowledge the use of Claude (https://claude.ai/) to assist with understanding implementation details related to the RAG pipeline, including chunking methods and retrieval workflows. The outputs were used to clarify concepts and support development, and I am able to explain and independently reproduce all work presented in this submission.

GenAI was also used for conceptual explanations and API clarifications for crawling and QA generation code. Final code for these modules was manually human-implemented to ensure full conceptual understanding. GenAI was also used to generate non-graded internal utility scripts for tasks such as facilitating manual annotations.

References

[1] Gao et al. (2023). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). ACL 2023.

[2] Xie et al. (2023). Investigating the Factual Knowledge Boundary of Large Language Models with Retrieval Augmentation. arXiv 2307.11019.

[3] BAAI/bge-small-en-v1.5. HuggingFace Model Hub.